C O N F I D E N T I A L S E C U R I T Y A S S E S S M E N T

Penetration Test Report

Orange Cyberdefense / SensePost Academy

Staged Web Application Challenge

Prepared for	Orange Cyberdefense (Client)
Prepared by	Sellwane Suzan Matsheka
Date	25 September 2026
Target	challenge.orangecyberdefense.academy
Highest level reached	Level 8 (Levels 0-7 completed; Level 8 reached and investigated)
Classification	Confidential

Confidentiality notice: This report is prepared for the Orange Cyberdefense assessment and contains security-sensitive information, including recovered credentials and secrets shown in full to evidence completion of each level. It should be handled and stored accordingly.

Table of Contents

1. Executive summary 3
2. Scope and methodology 4
3. Level-by-level walkthrough 5
Level 0: Reconnaissance (find the IP address) 5
Level 1: Client-side password in page source 5
Level 2: Base64-encoded secret in an external script 6
Level 3: Password assembled from multiple encodings 6
Level 4: Navigation logic behind a pop-up 7
Level 5: DNS TXT record enumeration 8
Level 6: Client-side encryption with a Sudoku-derived key 9
Level 7: Hidden route among decoy content (QR code) 9
Level 8: Reached and investigated (final route not identified) 10
4. Sudoku solution (Level 6) 12
5. Findings and remediation 13
Finding 1: Client-side and hardcoded authentication 13
Finding 2: Client-side cryptography (offline-decryptable) 13
Finding 3: Reversible obfuscation used as a security control 14
Finding 4: Information disclosure via public DNS records 14
Finding 5: Ineffective client-side restrictions 14
6. Conclusion 16
Appendix A : Tools used 16
Appendix B : References 16

1. Executive summary

Orange Cyberdefense provided a staged, web-based security challenge and asked for an assessment of how far it could be completed and how clearly the work could be documented. This section summarises the outcome and what it means in business terms.

The assessment worked through nine consecutive levels. The first eight were completed in full, and the ninth was reached and thoroughly investigated but not finished within the available time. In practical terms, this represents strong progress through the challenge, with each completed level documented step by step.

At almost every level, the protection relied on the user's own web browser rather than on a secure server. Passwords, secret keys, and the routes to the next level were all delivered to the browser and could be recovered by anyone willing to look. Where values were “scrambled,” they were scrambled with techniques that are designed to be easily reversed, not to keep a secret. In one case a set of login details for an administrator account was left on display.

Why this matters to the business

If these same design choices appeared in a live customer-facing application, the consequences would be direct and serious:

- Access controls could be bypassed. Anyone could reach protected areas without valid permission, because the checks happen where the attacker has full control.
- Sensitive information would be exposed. Secrets and credentials shipped to the browser can be read by any visitor, leading to unauthorised access and potential data breaches.
- Trust and compliance would be at risk. Exposed administrator credentials and weak protection of data are the kinds of issues that lead to regulatory findings, customer harm, and reputational damage.

The good news is that every issue observed is well understood and fixable with established practices. The single most important principle is that security decisions and secrets must live on the server, never in the browser, and that scrambling data is not the same as protecting it. Section 5 sets out prioritised, practical fixes immediate actions and longer-term improvements for each finding.

Summary of results

Measure	Result
Levels attempted	9 (Level 0 to Level 8)
Levels completed	8 (Level 0 to Level 7)
Highest level reached	Level 8
Distinct weaknesses identified	5 recurring themes (see Section 5)
Highest severity finding	High

2. Scope and methodology

In scope: The target host challenge.orangecyberdefense.academy, and the related host backup.orangecyberdefense.academy referenced in the first level. Testing was limited strictly to the challenge provided.

Timeframe: 24-25 September 2026, concluding before the 17:00 (+0200) submission deadline on 25 September 2026.

Methodology. Each level followed a consistent process aligned to industry practice (the PTES phases and the OWASP Web Security Testing Guide): understand the objective, perform reconnaissance and enumeration, analyse the findings and form a single hypothesis, attempt the solution, and record evidence before moving on. Page logic was reviewed directly from the command line, which allowed inspection independent of browser-based restrictions such as disabled right-click menus or interfering pop-ups. Findings are rated using the Common Vulnerability Scoring System version 3.1 (CVSS 3.1) and mapped to the OWASP Top 10 (2021).

Severity scale: Critical (9.0-10.0), High (7.0-8.9), Medium (4.0-6.9), Low (0.1-3.9), Informational (0.0).

Primary tools used: nslookup, dig, curl, base64, tr, rev, Node.js with the application's own crypto-js library, zbarimg, exiftool, strings, binwalk, nmap, and standard shell utilities, run from Kali Linux with Burp Suite available as a proxy.

3. Level-by-level walkthrough

Level 0: Reconnaissance (find the IP address)

Objective: Find the IP address of backup.orangecyberdefense.academy.

Approach and reasoning: DNS resolution was performed using the nslookup to identify the target IP.

Command executed: `nslookup backup.orangecyberdefense.academy`

Result: The host resolved to 127.127.1.1, within the 127.0.0.0/8 loopback range an unusual choice for a public name, noted as an observation.

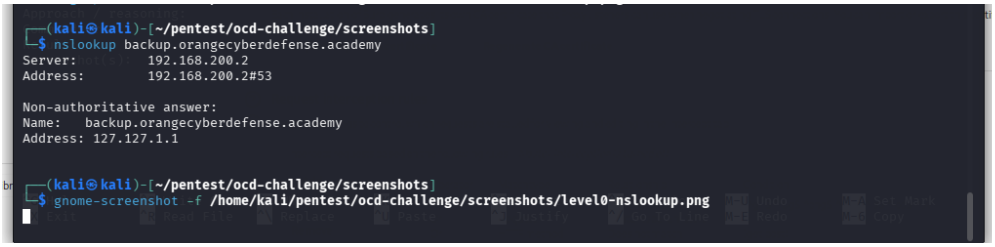


Figure 1: level 0- DNS lookup resolving backup.orangecyberdefense.academy to 127.127.1.1

Level 1: Client-side password in page source

Objective: Enter the correct password to continue.

Approach and reasoning: A password prompt on a simple page validates the value in the browser. Rather than guessing, the full page source was retrieved and the client-side logic inspected.

Command executed: `curl "https://challenge.orangecyberdefense.academy/level1-e137ac7cd5110d469a631a1cc2393227.html"`

Result: The password was hardcoded in a JavaScript function as Frame-Zoo-Fog-Package2-Choose. The next level's URL is built from md5(password). (Relates to Finding 1.)



Figure 2: level 1 hardcoded password found in the page source



Figure 3: level 1 progress

Level 2: Base64-encoded secret in an external script

Objective: Find the secret key to continue.

Approach and reasoning: The checking logic had been moved into external scripts. The referenced JavaScript was retrieved and found to Base64-encode the user's input and compare it to a fixed string, meaning the password is the Base64 decode of that string. Several decoy functions were present but never called.

Command executed: `curl "https://challenge.orangecyberdefense.academy/level2-checkdecoded.js"`

Command executed: `echo "YmFzZV82NF9pc19lbmNvZGluZw==" | base64 -d`

Result: The secret key decoded to base_64_is_encoding. (Relates to Finding 3.)

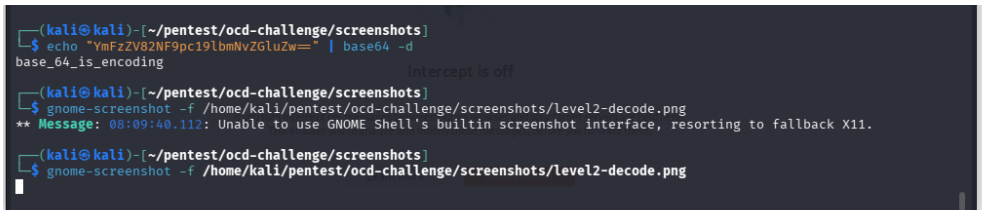


Figure 4: level 2 Base64 decode producing the secret key base_64_is_encoding

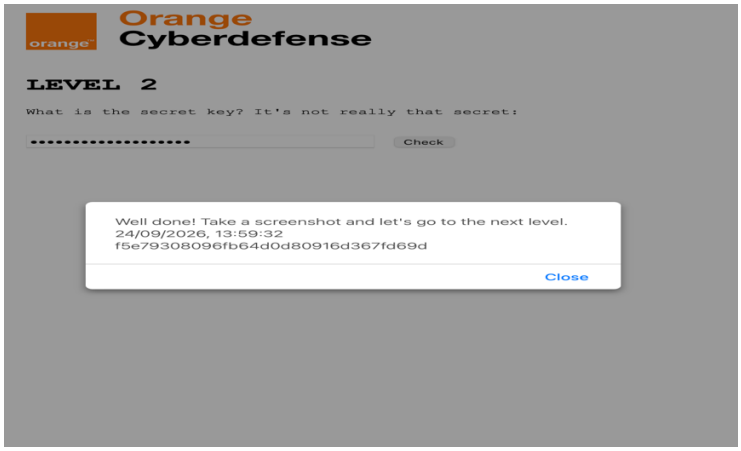


Figure 5: level 2 progress

Level 3: Password assembled from multiple encodings

Objective: Provide the correct password to continue.

Approach and reasoning: The password was assembled from three parts in client-side JavaScript: a Base64-decoded string (“The_Obstacle_”), a run of ASCII character codes (“isthe”), and an underscore followed by three characters extracted by index from a longer Base64-decoded sentence (“_way”). Each value was decoded and the exact characters extracted by index using shell utilities, then the parts were joined. The page also disabled right-click, which had no effect because the source was read from the command line.

command executed:
```
a="The_Obstacle_"
b="isthe"
text="There is only one way to go. That is forward. Good luck. The Obstacle is the way."
c="_${text:18:1}${text:41:1}${text:12:1}"
echo "$a$b$c"
```

Result: The password was The_Obstacle_isthe_way. (Relates to Findings 1, 3 and 5.)



Figure 6: level 3 password reconstructed from Base64, character codes, and index extraction.

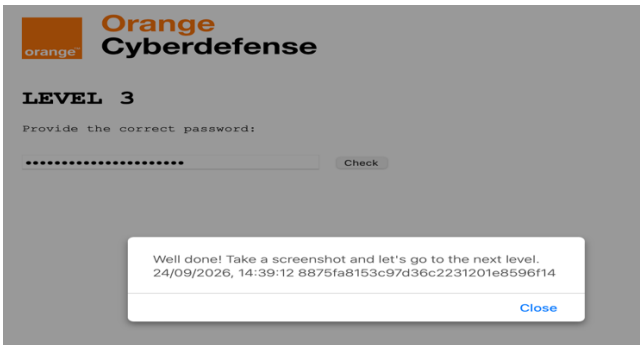


Figure 7: level 3 progress

Level 4: Navigation logic behind a pop-up

Objective: Click through to continue to the next level.

Approach and reasoning: The page loaded a pop-up that re-triggered on OK or Cancel, designed to trap interaction in the browser. There was no password to find; the Next button simply redirected to the next level. The source was reviewed and the next URL extracted directly, bypassing the pop-up.

Command executed: `curl "https://challenge.orangecyberdefense.academy/level4-The_Obstacle_isthe_way.html"`

Result: The next URL was found in the redirect logic: level5-56ff181fdf03832f9a001d1b98568c98.html. (Relates to Finding 5.)

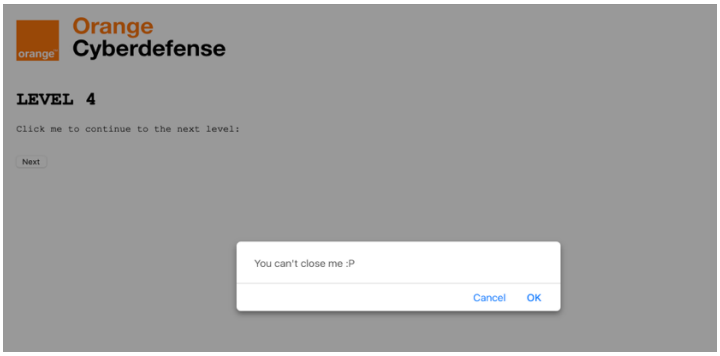


Figure 8: level 4 pop-up presented in the browser



Figure 9: level 4 next-level URL found in the page source

Level 5: DNS TXT record enumeration

Objective: Find the TXT record for challenge.orangecyberdefense.academy.

Approach and reasoning: This level shifted to server-side DNS enumeration. A TXT record is a text field attached to a domain, and it was queried directly.

Command executed: `dig TXT challenge.orangecyberdefense.academy`

Result: The TXT record contained xP0UfmVf7u. The domain is served by AWS Route 53 nameservers. (Relates to Finding 4.)

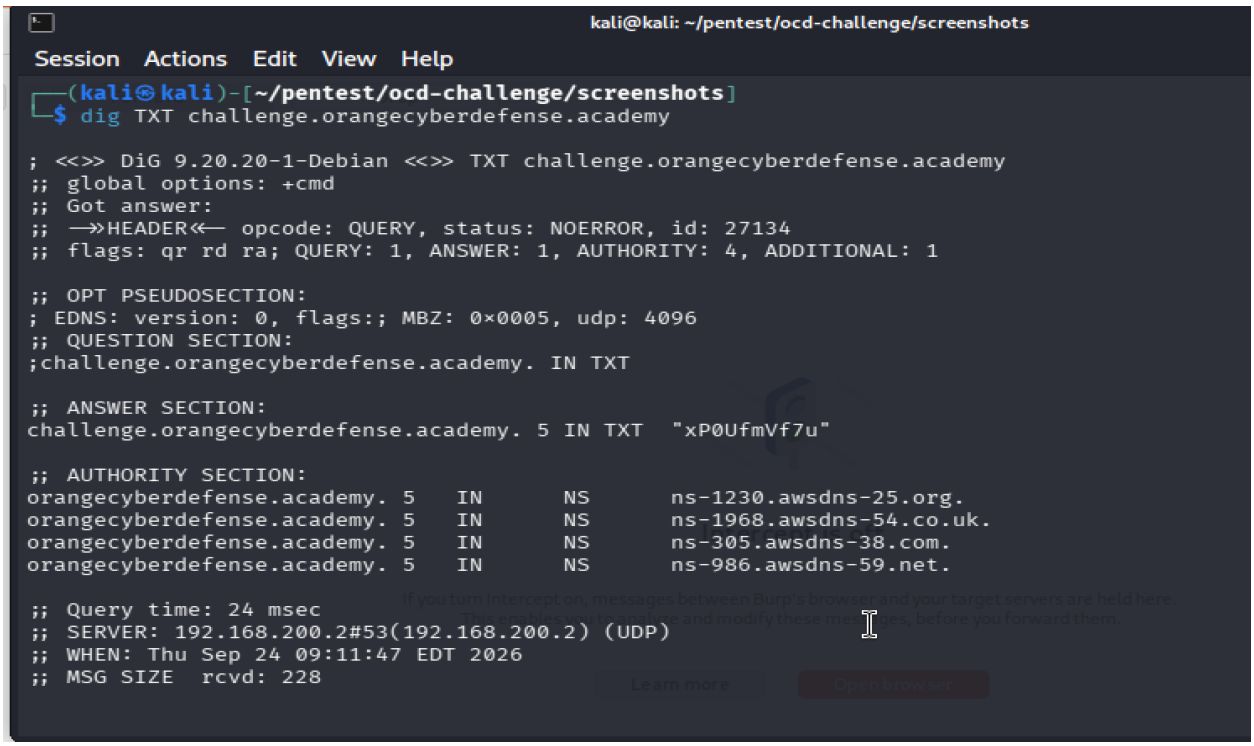


Figure 10: level 5 DNS TXT record query returning xP0UfmVf7u

Level 6: Client-side encryption with a Sudoku-derived key

Objective: Find and decode the encrypted string using a key derived from a Sudoku puzzle.

Approach and reasoning: The application used a client-side AES routine, with the encrypted text and validation logic all delivered to the browser. The key was derived from a Sudoku puzzle whose five red-outlined cells give the key digits once solved. The puzzle was solved and the red cells read (4, 3, 8, 7, 2). Because manual ordering and openssl did not match the application's encryption, the application's own crypto library was run in Node.js to test every ordering against its success check, which identified the correct key.

Command executed: `curl -o crypto-js.js "https://challenge.orangecyberdefense.academy/crypto-js.js"`
`node solve6.js  # tested key orderings using the application's own crypto library`

Result: The key was 34278 and the decrypted value was w3_mov3_2_the_future. The Sudoku solution appears in Section 4. (Relates to Finding 2.)

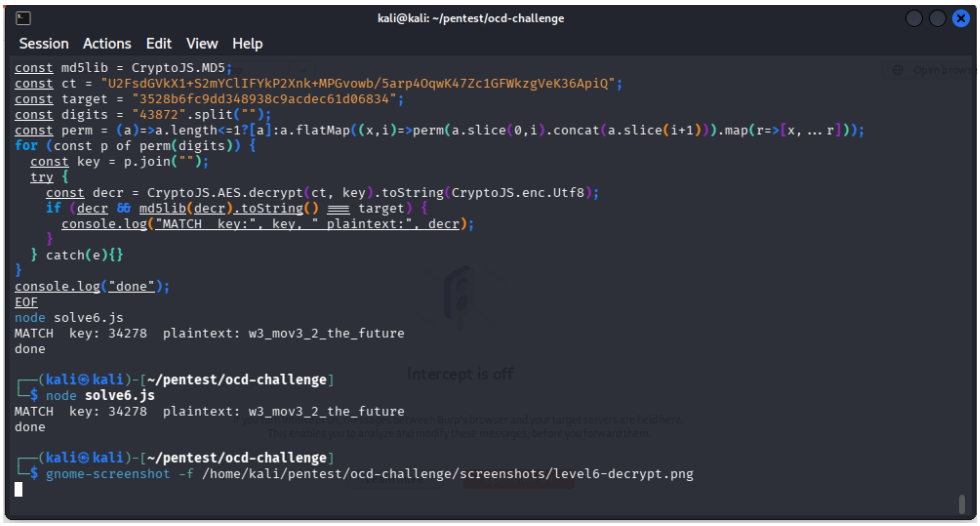


Figure 11: level 6 decryption run identifying key 34278 and plaintext w3_mov3_2_the_future

Level 7: Hidden route among decoy content (QR code)

Objective: Find the hidden route to the next level within the page content.

Approach and reasoning: The page held deliberate decoy content plaintext quotes, ROT13-encoded quotes (confirmed as ordinary quotes by decoding them), encrypted strings, and images. One image was a QR code encoding the next URL, which was decoded to obtain the address.

Command executed: curl -o owfjwlf77.png "https://challenge.orangecyberdefense.academy/owfjwlf77.png"
curl -o 23jkhqe30.png "https://challenge.orangecyberdefense.academy/23jkhqe30.png"

Images: `zbarimg 23jkhqe30.png owfjwlf77.png`

Result: The QR code decoded to the Level 8 page URL. (Obfuscation exercise; relates to Finding 2.)
Decoded QR value: level8-b65079c27a33adc0b65d7038af7071ab.html

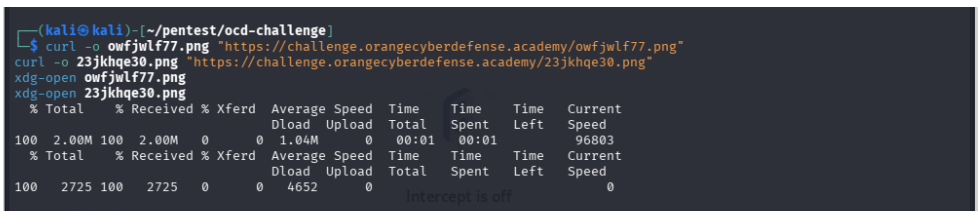


Figure 12: level 7 QR code hidden among decoy content

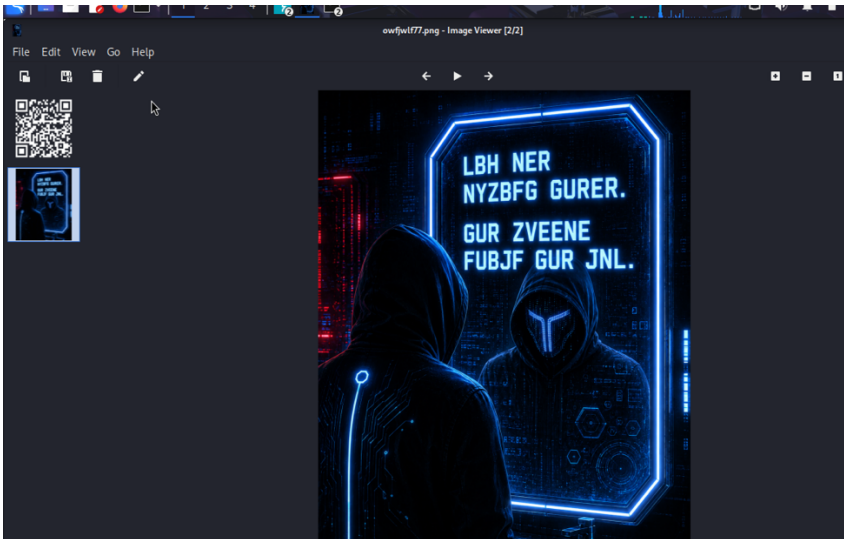


Figure 13: level 7 decoded QR code revealing the level 8 URL

Level 8: Reached and investigated (final route not identified)

Objective: Progress past a login-themed image containing credentials and a URL.

Approach and reasoning: The page showed an image of a login screen with the username sa, a ROT13-encoded password (decoding to Summerattwentysix), and a URL. The level was investigated.

- The sa username was recognised as the Microsoft SQL Server administrator account; a port scan showed only web ports open behind Amazon CloudFront, so no live service was available to authenticate against.
- The image was checked for hidden data; the metadata contained only a decoy message and no data was embedded.
- A second image contained a further clue: a ROT13 message decoding to “You are almost there. The mirror shows the way,” indicating a value must be reversed.
- The reversal was tested in multiple ways (reversed values and mirrored paths); none produced a working next-level address before the deadline.

Result: All decodable clues were recovered and the intended mechanism (reversal) was identified, but the final path was not found in time. Highest level reached: 8. (Relates to Findings 1 and 3.)

Credentials recovered: sa / Summerattwentysix (ROT13-decoded)

Reversal candidates tested: 1283b4a9037fc683f08d4098737671c9, xisytnewttaremmuS (all returned HTTP 404)

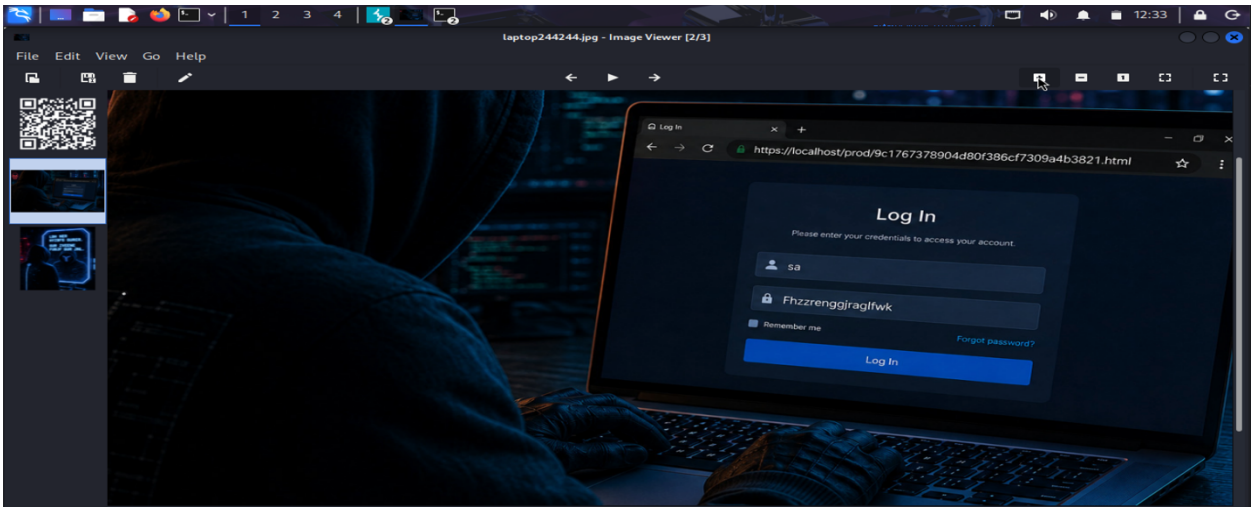


Figure 14: level 8 login screen showing username sa and a ROT13- encoded password

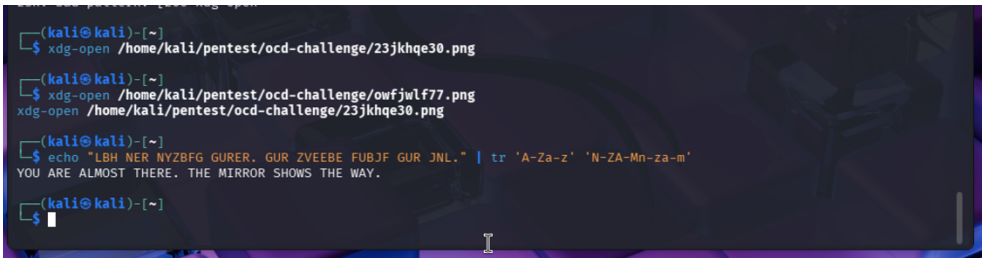


Figure 15: level 8 neon clue decoding to "YOU ARE ALMOST THERE. THE MIRROR SHOWS THE WAY"

4. Sudoku solution (Level 6)

As requested in the challenge, the solved Sudoku is included. The five red-outlined cells, read from the completed grid, provided the digits used to build the Level 6 key.

Red cell values: R1C1 = 4, R4C6 = 3, R7C4 = 8, R7C7 = 7, R8C2 = 2 (digits 4, 3, 8, 7, 2).

Correct key ordering: 34278, which decrypted the value to w3_mov3_2_the_future.

					6			
				5		6		4
3					4			9
			1	8			7	
	9						3	
8						5		
9	6	4						
		3	5		7			
		5	2					

Figure 16: original grid

4	1	8	9	2	6	3	5	7
2	7	9	3	5	8	6	1	4
3	5	6	7	1	4	2	8	9
5	4	2	1	8	3	9	7	6
6	9	1	4	7	5	8	3	2
8	3	7	6	9	2	5	4	1
9	6	4	8	3	1	7	2	5
1	2	3	5	6	7	4	9	8
7	8	5	2	4	9	1	6	3

Figure 17: solved sudoku

5. Findings and remediation

The levels are puzzles rather than a live production system, so severities are assessed as if these design patterns appeared in a real application. Each finding includes a CVSS 3.1 rating, the business impact, and prioritised remediation split into short-term (immediate) and long-term (durable) actions.

#	Finding	Severity	CVSS
1	Client-side / hardcoded authentication	High	7.5
2	Client-side cryptography (offline-decryptable)	High	7.5
3	Reversible obfuscation used as protection	Medium	5.3
4	Information disclosure via public DNS	Medium	5.3
5	Ineffective client-side restrictions	Low	3.1

Finding 1: Client-side and hardcoded authentication

Severity	CVSS 3.1	OWASP 2021	Affected levels
High	7.5	A07 Identification and Authentication Failures	Levels 1, 3, 8

CVSS vector: `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`

Description. Passwords and login logic were embedded in code delivered to the browser. Anyone viewing the page source could read the credentials directly, and in one level a full set of administrator login details was on display.

Business impact. An attacker can authenticate as any user, including administrators, without authorisation. This can lead to full account takeover, access to restricted functionality, and exposure of any data those accounts can reach a direct route to a data breach.

Remediation: short term:

- Remove all credentials and authentication logic from client-side code and page source immediately.
- Rotate any credentials that have been exposed, especially administrator accounts such as “sa”.
- Review whether the exposed accounts have been used from unexpected locations.

Remediation: long term:

- Perform all authentication on the server; the browser should only submit input, never hold the answer.
- Store passwords only as salted hashes, never in recoverable form.
- Adopt secure development practices and code review to prevent secrets entering client-side code, supported by automated secret-scanning in the build pipeline.

Finding 2: Client-side cryptography (offline-decryptable)

Severity	CVSS 3.1	OWASP 2021	Affected levels
High	7.5	A02 Cryptographic Failures	Level 6

CVSS vector: `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`

Description. Encryption was performed entirely in the browser, with the encrypted data, the algorithm, and the success check all delivered to the client. This allowed the value to be decrypted offline using the application's own code, without needing the real secret.

Business impact. Sensitive data protected only by client-side encryption can be recovered by any user, defeating the purpose of encrypting it. In production this would expose confidential data and any secrets handled the same way.

Remediation: short term:

- Stop relying on browser-side encryption to protect any sensitive value.
- Treat any data currently protected this way as exposed and reissue affected secrets.

Remediation: long term:

- Move encryption and decryption of sensitive data to the server, with keys held securely server-side and never sent to the browser.
- Serve sensitive content only over authenticated, server-side channels.

Finding 3: Reversible obfuscation used as a security control

Severity	CVSS 3.1	OWASP 2021	Affected levels
Medium	5.3	A02 Cryptographic Failures	Levels 2, 3, 7, 8

CVSS vector: `AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`

Description. Secret values were protected with encoding schemes such as Base64 and ROT13, and with simple reversal. These techniques change how data looks but are trivially reversible by anyone, so they provide no real protection.

Business impact. Any value "protected" this way is effectively public. If used for sensitive data or access tokens in production, it would allow attackers to recover secrets and bypass intended controls, undermining confidentiality.

Remediation: short term:

- Identify any place where encoding is currently relied on to keep a value secret and treat those values as already exposed.
- Where such values grant access, invalidate and reissue them.

Remediation: long term:

- Use encoding only for data formatting, never for protection. Protect genuine secrets with vetted, strong encryption and proper key management.
- Train developers on the distinction between encoding, hashing, and encryption.

Finding 4: Information disclosure via public DNS records

Severity	CVSS 3.1	OWASP 2021	Affected levels
Medium	5.3	A01 Broken Access Control / Information Exposure	Level 5

CVSS vector: `AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`

Description. A secret value was stored in a public DNS TXT record. DNS records are queryable by anyone on the internet, so any value placed there is effectively public.

Business impact. Sensitive information published in DNS can be harvested by attackers for reconnaissance or direct use, contributing to a broader compromise.

Remediation: short term:

- Review DNS records and remove any that contain sensitive or secret values.
- Confirm no credentials, keys, or internal details are exposed in public records.

Remediation: long term:

- Establish a process for reviewing DNS entries before publication, and keep secrets in managed secret stores rather than public records.

Finding 5: Ineffective client-side restrictions

Severity	CVSS 3.1	OWASP 2021	Affected levels
Low	3.1	A05 Security Misconfiguration	Levels 3, 4

CVSS vector: `AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N`

Description. The application used browser-side restrictions such as a disabled right-click menu and a blocking pop-up to deter inspection. These affect only casual users and are bypassed instantly by anyone reading the page from the command line.

Business impact. These measures create a false sense of security while providing none. They do not stop a determined attacker and can mask the absence of real controls.

Remediation: short term:

- Do not rely on client-side restrictions as a security measure; document that they are cosmetic only.

Remediation: long term:

- Enforce every security decision on the server. Use client-side behaviour only for user experience, never for protection.

6. Conclusion

Levels 0 through 7 were completed and Level 8 was reached and thoroughly investigated, for a highest level reached of 8. Progress required a broad range of techniques: DNS enumeration, client-side source review, several decoding and cryptographic operations, and image and host analysis, all documented level by level.

The consistent lesson is that security enforced in the browser, and secrets hidden with reversible encoding, provide no real protection. Every value that reached the browser was recoverable and every browser-side restriction was bypassable. These are common, serious weaknesses. The remediation in Section 5 sets out how to address each one, with the overarching principle being simple: enforce security on the server, and protect secrets with real cryptography rather than obscurity.

Appendix A : Tools used

- DNS: nslookup, dig
- HTTP and source review: curl, Burp Suite
- Encoding and decoding: base64, tr (ROT13), rev
- Cryptography: Node.js with the application's crypto-js library, openssl (comparison), md5sum
- Image and file analysis: zbarimg, exiftool, strings, binwalk, ImageMagick
- Host enumeration: nmap

Appendix B : References

The following standards, frameworks, and resources informed the methodology, severity ratings, and remediation guidance in this report.

- OWASP Foundation. OWASP Top 10 (2021). https://owasp.org/Top10/
- OWASP Foundation. Web Security Testing Guide (WSTG). https://owasp.org/www-project-web-security-testing-guide/
- FIRST. Common Vulnerability Scoring System (CVSS) v3.1 Specification. https://www.first.org/cvss/v3-1/
- The Penetration Testing Execution Standard (PTES). http://www.pentest-standard.org/
- OWASP Foundation. Cryptographic Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html
- OWASP Foundation. Authentication Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- MITRE. CWE-798: Use of Hard-coded Credentials. https://cwe.mitre.org/data/definitions/798.html
- MITRE. CWE-311: Missing Encryption of Sensitive Data. https://cwe.mitre.org/data/definitions/311.html